

Motion Planning Assignment

Laura Scigliano
s331082@studenti.polito.it

Abstract—This report presents an analysis of the performance differences between Dijkstra’s algorithm and the A* algorithm in pathfinding over geographic road networks. The study analyzes how the computational efficiency and solution quality vary between the two algorithms as the size and complexity of the underlying graph increase, through real-world map. A secondary focus is placed on evaluating the impact of different heuristics within A*, on both iterations number and path cost. Results show that A* consistently reduces the number of iterations compared to Dijkstra, with this advantage becoming more pronounced in larger networks, while the path cost differences depend on the heuristic choice.

I. INTRODUCTION

Pathfinding algorithms play a fundamental role in applications involving navigation and routing over road networks, such as in autonomous vehicles and GPS-based systems. Among the most widely used algorithms in this context are Dijkstra’s algorithm and the A* search algorithm. Both aim to find the shortest path between a source and a destination node within a graph, but they do so using different approaches.

Dijkstra’s algorithm guarantees the optimal path by exploring all possible routes in increasing order of cost, without any directional bias. This makes it reliable but potentially computationally expensive, especially in large or complex networks.

The A* algorithm extends Dijkstra’s approach by introducing a heuristic function that estimates the cost from the current node to the destination. This heuristic allows A* to prioritize exploration in the most promising directions, often reducing the number of nodes visited and improving efficiency, particularly in larger graphs.

This project aims to compare the performance of Dijkstra and A* in real-world scenarios using road networks obtained with the `osmnx` library. It also investigates how different heuristics in A* affect computational cost and path quality.

II. THE ROLE OF THE HEURISTIC IN A*

A key component that differentiates the A* algorithm from Dijkstra is the use of a heuristic function. This function, typically denoted as $h(n)$, provides an estimate of the cost from a given node n to the destination. Combined with the actual cost from the start node, $g(n)$ (g-score), A* computes a total score $f(n) = g(n) + h(n)$ (f-score), which guides the search process.

The heuristic has the role to prioritize nodes that are likely to lead more directly to the destination node, reducing the number of nodes explored and improving performance, especially in large graphs. The balance between the heuristic $h(n)$ and the $g(n)$ determines how A* explores the graph

and directly impacts both the path quality and computational efficiency. Several behaviors can emerge based on the relative magnitude of $h(n)$ ¹:

- $h(n) = 0$: A* behaves exactly like Dijkstra’s algorithm.
- $h(n)$ is smaller or much smaller than $g(n)$: The heuristic consistently underestimates the actual cost, which is admissible and ensures the optimality of the solution, but this leads to more node expansions and longer execution times.
- $h(n)$ is approximately equal to the true cost to the goal: This is the ideal scenario. The algorithm expands only the nodes necessary along the optimal path, minimizing computation time while maintaining correctness. Although such precision is rare in practice.
- When $h(n)$ is greater than $g(n)$: The heuristic starts to dominate the decision-making process, directing the search more aggressively toward the goal. This can reduce the number of explored nodes and speed up execution, but if the heuristic is not admissible, the algorithm may fail to find the optimal path.
- $h(n) \gg g(n)$: A* relies almost entirely on the heuristic estimate. While this can greatly accelerate the search, it also significantly increases the likelihood of suboptimal paths, especially in complex or irregular graphs.

In summary, the design and tuning of the heuristic involve a trade-off between accuracy and speed.

In this study, two heuristic functions were used: the Manhattan distance and the Haversine distance.

The Manhattan distance estimates the cost based on movement constrained to horizontal and vertical steps, making it particularly suitable for grid-like environments such as urban street layouts where travel occurs along orthogonal paths.

The Haversine distance, on the other hand, calculates the great-circle distance between two points on the Earth’s surface, taking into account the Earth’s curvature. This makes it a more accurate heuristic for geographic routing tasks involving large-scale road networks, where the actual shortest path approximates the arc over the globe rather than a simple straight line.

In code implementation, the heuristics were computed taking into account not only the geometric distance but also an estimated travel time, reflecting the fact that edge weights represent time rather than simple distance. Specifically, the Haversine heuristic was calculated using the `great_circle`

¹Amit Patel, “A* Heuristics,” Game Programming, Stanford University, accessed May 18, 2025, <https://theory.stanford.edu/~amitp/GameProgramming/Heuristics.html#linear-exact-heuristic>.

function from the `geopy.distance` module, which computes the shortest path over the Earth’s surface between two coordinates in kilometers. The estimated time was then obtained by dividing this distance by an average speed, which corresponds to the arithmetic mean of all the maximum speeds (`maxspeed`) associated with the edges of the graph, and converting the result into seconds, as follows:

$$h_{\text{Haversine}}(n) = \frac{\text{great_circle}(\text{coord}_1, \text{coord}_2).\text{km}}{\text{average_speed}} \times 3600$$

Similarly, the Manhattan heuristic was adapted to reflect travel time by multiplying the Manhattan distance in kilometers by a scaling factor $D = 1$, then dividing by the average speed and converting to seconds:

$$h_{\text{Manhattan}}(n) = \frac{D \times \text{distance_km}}{\text{average_speed}} \times 3600$$

This approach aligns with the edge weight definition used in the graph, where each edge’s weight corresponds to travel time calculated as

$$w(e) = \frac{\text{length}(e)/1000}{\text{maxspeed}} \times 3600,$$

with length measured in meters, maxspeed in kilometers per hour, and the factor 3600 converting hours to seconds.

III. RESULTS ANALYSIS AND COMPARISON

The experiments were conducted on two different maps: Turin, Italy (the larger map), and Piedmont, California (the smaller one). To ensure consistent and comparable results, each algorithm was executed ten times, each time with different start and goal nodes.

Table I reports the results obtained on the Piedmont map for three variants: Dijkstra’s algorithm, A* with the Haversine heuristic computed using an average speed of approximately 37.7 km/h, and A* with Haversine using a constant speed of 50 km/h (Haversine 50)). Table II shows the results obtained on the Turin map using the same three variants saw before, and additionally, A* using the Manhattan heuristic computed with an average speed of approximately 41 km/h.

The results on the Piedmont map indicate that even on relatively small-scale maps, A* significantly reduces the number of iterations compared to Dijkstra. In the second test case, the iteration count decreased by up to 75%, while still yielding the exact same path as Dijkstra, as confirmed by the identical total path cost. However, A* did not always return the same optimal paths. For instance, in test case 3, although the iteration count was reduced by approximately 21%, the best path identified by A* differed from Dijkstra’s. This discrepancy is due to the heuristic not perfectly estimating the actual remaining cost to the goal, causing A* to select a suboptimal, but still very close, path. Out of the ten test cases, only three resulted in different paths compared to Dijkstra’s (test cases 3, 6, and 10). The differences in total path cost, however, were minimal: under 2% in test cases 3 and 10, and approximately 10% in test

case 6. The version of A* using the “Haversine 50” did not yield substantially better results than the version using the average speed, even though using a higher speed reduces the computed heuristic values, effectively making the heuristic less dominant.

When comparing the results of the four algorithms applied to the map of Turin, it becomes evident that A* significantly reduces the number of iterations up to 88% (in path 10) compared to Dijkstra algorithm. Although the resulting path is not always exactly the same, the total cost differs by only 0.09% between A* Haversine and Dijkstra. In this map, more paths deviate from the optimal solution found by Dijkstra than in Piedmont one, specifically 6 out of 10 (paths 1, 2, 6, 7, 9, and 10), but the cost difference never exceeds 1.7%.

This indicates that, while there is a slight loss of accuracy in more cases than observed with the Piedmont map, the reduction in accuracy remains minimal. At the same time, the algorithm gains substantial speed improvements, with reductions in iteration count reaching up to 88% (in path 10 A* executes 9,770 less than Dijkstra).

Compared to standard A*, the “Haversine 50” variant produced better results in terms of path accuracy: 8 out of 10 paths exactly matched those found by Dijkstra, and the remaining ones deviated by no more than 1.7% in cost. However, this came at the expense of higher iteration counts. The reason is that using a speed of 50 km/h (higher than the average speed of 41 km/h) results in lower h-scores, which causes A* to consider a larger number of alternative routes, as clearly shown in Figure 2 (c).

Finally, the A* implementation using the Manhattan heuristic produced results nearly identical to A* with the Haversine heuristic and average speed. The total cost of the optimal paths was the same of A* Haversine in all cases, although the number of iterations was slightly higher, up to 52 additional iterations.

IV. CONCLUSION

The experiments show that with a well-chosen heuristic, such as the Haversine or Manhattan distance adapted to time-based weights, A* can maintain a high level of accuracy while offering significant computational savings. Nonetheless, careful attention must be paid to the definition and scaling of the heuristic, as an overly optimistic or poorly tuned heuristic may lead to less optimal paths.

In conclusion, the comparison between Dijkstra and A* highlights the clear advantage of A* in terms of computational efficiency, especially as the size and complexity of the map increase. While both algorithms are capable of finding optimal paths, A* consistently requires fewer iterations, making it a more suitable choice for large or dense networks where performance is critical and a slight loss in path accuracy is acceptable in exchange for faster computation.

TABLE I: Confronto tra Dijkstra e A* con diverse euristiche su Piedmont

Test #	Dijkstra		A* (Haversine)		A* (Haversine 50)	
	Iter	Weight	Iter	Weight	Iter	Weight
1	12	26.03	5	26.03	7	26.03
2	275	177.17	71	177.17	139	192.67
3	348	448.59	272	449.74	301	448.59
4	280	216.76	136	216.76	185	216.76
5	149	223.24	68	223.24	87	223.24
6	175	169.23	85	187.37	124	186.22
7	84	151.89	34	151.89	41	151.89
8	145	114.58	24	114.58	40	114.58
9	275	240.69	147	240.69	188	240.69
10	318	364.57	185	370.76	293	378.60

TABLE II: Confronto tra Dijkstra e A* con diverse euristiche su Torino

Test #	Dijkstra		A* (Haversine)		A* (Haversine 50)		A* (Manhattan)	
	Iter	Weight	Iter	Weight	Iter	Weight	Iter	Weight
1	7118	679.22	1327	686.79	2220	679.22	1347	686.79
2	4769	331.10	136	331.86	424	331.10	135	331.86
3	7700	491.47	821	491.47	1392	491.47	824	491.47
4	10252	764.25	1120	777.43	2154	764.25	1125	777.43
5	3204	365.69	499	365.69	848	365.69	503	365.69
6	9889	631.10	757	634.22	2362	631.10	766	634.22
7	10281	726.72	1525	728.20	4005	727.22	1555	728.20
8	11519	920.46	4779	920.46	6447	920.46	4831	920.46
9	5703	599.96	1033	607.35	2487	607.62	1049	607.35
10	11069	673.06	1299	673.63	2555	673.06	1314	673.63



(a) Dijkstra explored routes path 3



(b) A* Haversine explored routes path 3



(c) A* Haversine 50 explored routes path 3



(d) Dijkstra best path for path 3



(e) A* Haversine best path for path 3

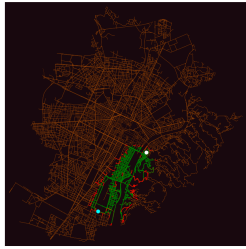


(f) A* Haversine 50 best path for path 3

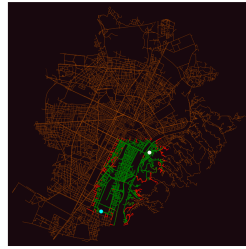
Fig. 1: Comparison of the paths computed by Dijkstra and A* with different heuristics on Piedmont



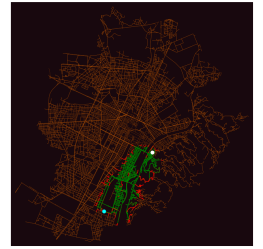
(a) Dijkstra explored routes path 3



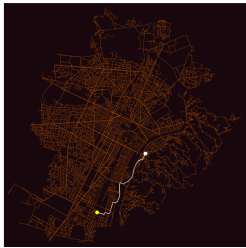
(b) A* Haversine explored routes path 3



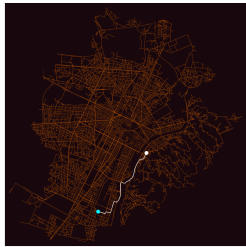
(c) A* Haversine 50 explored routes path 3



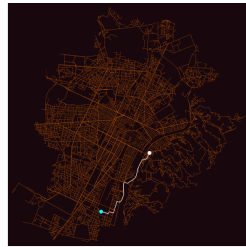
(d) A* Euclidean explored routes path 3



(e) Dijkstra best path for path 3



(f) A* Haversine best path for path 3



(g) A* Haversine 50 best path for path 3



(h) A* Euclidean best path for path 3

Fig. 2: Comparison of the paths computed by Dijkstra and A* with different heuristics on Turin